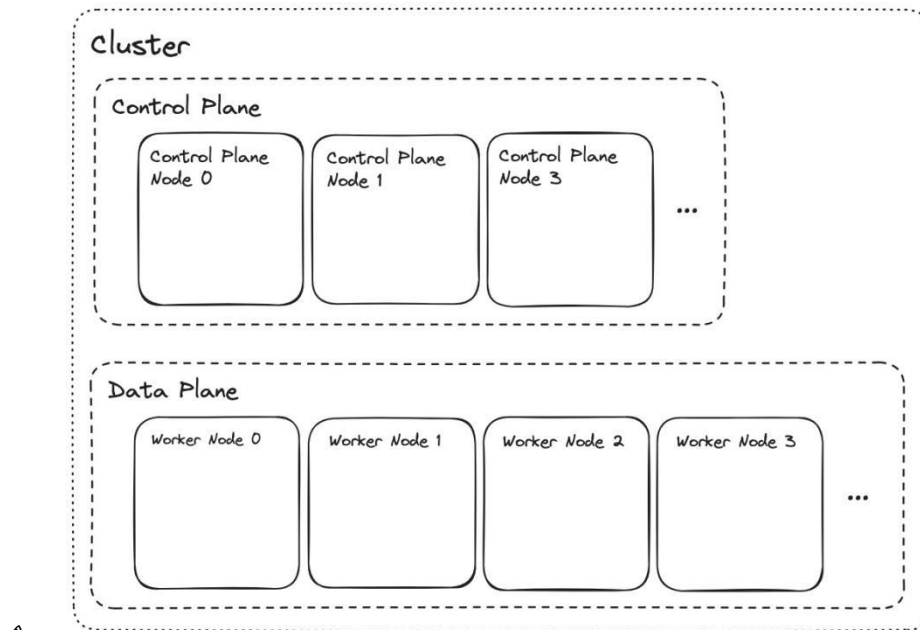
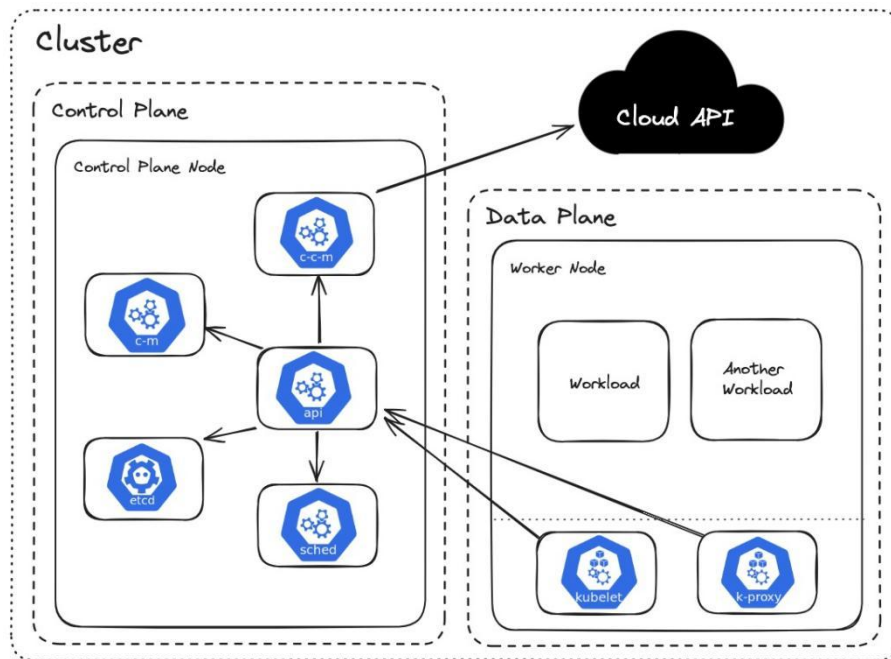


- GitHub: <https://github.com/sidpalas/devops-directive-kubernetes-course/tree/main>
- Planes and Nodes



- Nodes are either bare metal or virtual machines that is running the applications.
- Nodes are divided in 2 parts, one is **Control Plane** and another is **Data Plane**.
- The planes combined forms **Cluster**.
- Control Plane (Brain of Kubernetes)
 - It is where the system components run.
 - It makes decisions, manages cluster, schedules the workloads.
 - It decides **where pods should run, keeps track of cluster state, handles API requests**.
 - It includes components like **API Server, Scheduler, Controller Manager, etcd** (database).
- Data Plane (Workers)
 - It is where the user-end applications run.
 - It actually runs your applications (pods/containers)
 - Each worker node runs **Pods** (your app), **Kubelet** (talks to control plane), **Container runtime** (**Docker/containerd**)
- The flow will be like below:
 - You deploy an app (**kubectl apply**)
 - Control plane decides which worker node should run it.
 - Worker node runs the container (your app)
 - Control plane keeps checking if everything is healthy.

- ⌘ Analogy:
 - ⌘ Control plane: manager
 - ⌘ Data plane: Employee
 - ⌘ Pods: Tasks
 - ⌘ Manager assign tasks == Employee executes == Manager monitors.
- The below is the zoomed in version of Control plane node and Data plane node



⌘ Control Plane Node

c-c-m (cloud controller provider)

- ⌘ It connects Kubernetes with the cloud provider (AWS, GCP, Azure)

api (Api Server)

- ⌘ Entry point of kubernetes.
- ⌘ Every command (kubectl, UI, scripts) comes here.
- ⌘ Validates requests
- ⌘ Talks to other components.

etcd (Database)

- ⌘ It is the database of Kubernetes.
- ⌘ Stores entire cluster data like, pods info, nodes info, configs.

sched (Scheduler)

- ⌘ Decides where pods to run.
- ⌘ Choose the best worker node based on CPU/Memory, Availability.
- ⌘ It decides, “where should run?”

c-m (Controller Manager)

- Maintains desired state.
- **Its actually a container that contains all the controllers.**
- **Controllers are not any separate machine, these are just logic/processes running inside CM.**
- Pods crashes → recreate it
- Nodes down → reschedule pods
- You can think it as a *Auto Healing System*.
- It decides “**what should exist?**”

• Data Plane Node

kubelet

- It runs on every worker node.
- Talks to API server.
- Ensures pods are running.
- What it does?
Pods assigned to its node.
Pull docker image, starts the container.
Monitors health, restart if crashes.
- **Scheduler assigns Pods, Kubelet runs that.**

Kube Proxy

- Handles networking inside node.
- Routes traffic to correct pod.
- What it does?
Maintains network rules (iptables/ipvs)
Enable service → Pod communication.

Pods (Workloads)

- These are the actual applications.
- **Pods are just logical units, the containers are actual running processes.**
- Each pod → 1 running instance.
- Inside each pod: container (your app), shared network, optional storage.

➤ Now the question is, **kubelet** restarts the **containers (not pods)** if crashes, then why CM exists.

- Lets say one node goes down, it means the kubelet present inside that node is also gone.
- Now CM checks if the required number of pods are running or not (according to the desired state)

- ⌘ If not, then it creates new pods objects (not containers)
- ⌘ After that, scheduler decides on which node that pods will be running.
- ⌘ Now, **kubelet** present inside each node (which is chosen for a pod), will run the containers inside the specific pods.
- ⌘ One pod can run multiple *containers*, but they will share same ip and resources, it is advanced if you want to use logging or some functionality like that.
- ⌘ **Kubelet restart the containers if crashes, because pods are just objects which is created by CM inside which Containers are run.**

➤ CM and Schd

- ⌘ **CM:** We need 4 pods (create pods objects)
 - ⌘ **SCHD:** Where should these pods go? Pod-1 → Node-1, Pod-2 → Node-2 ..etc
 - ⌘ **Kubelet:** Runs on each worker node.
- It sees, Pod-1 is assigned to my node, then pull docker image, starts container and keep running.

➤ Nodes vs Pods

- ⌘ Pod is application level, it is the one which runs the container.
- ⌘ Node, you can think of a computer, that can run multiple pods inside it.
- ⌘ Now the question is, if a Node can run multiple Pods, then why do we need multiple Worker Nodes?
 - ⌘ First thing is high availability and avoiding single point of failure.
 - ⌘ 2nd reason can be cpu and ram.

Lets say you have one node having 2 cores and 4 GB of ram.

You have to run 4 pods, each will use 1 core and 2 GB of ram.

Here, you need 2 worker node to run 4 pods.

Pod-1, Pod-2 → Node-1, Pod-3, Pod-4 → Node-2
- ⌘ **You define how many app instances you want, and Kubernetes automatically creates pods and distributes them across nodes based on available resources and load balancing.**

Node, Kubernetes, VMs (provided by Cloud providers or Bare metal)

- You must be thinking what is the work of **c-c-m** and if all the nodes and things are there inside Kubernetes cluster, then why do we need cloud provider's VMs.
- First, What is a **Node**?
 - ⌘ Its just a machine that runs multiple processes (services).
 - ⌘ Its not necessary that the control plane processes and data plane processes will run in different Node, they can run in a single Node as well.
 - ⌘ Lets say you install **all inside one VM**.
 - ⌘ It'll have control plane processes like: **c-c-m** (only if VM is provided by a cloud provider), **c-m**, **etcd**, **sched**, **api**.
 - ⌘ Also, it'll have data plane processes like: **Pods**, **kubelet**, **k-proxy**.
 - ⌘ Now, when you tell it the number of instances you want to create, same like before, **c-m** will create pods objects, **sched** will assign the pods to the node.
As we have only node inside which all processes are running, so **sched** will assign this node only, for all the pods.
So, all the pods will be running inside the same VM.
- Now, as we saw before, **c-m** checks if pods are gone (when node goes down) then it creates new pods objects to get the desired state and then **sched** will assign those pods to some nodes. So **"Are the Nodes get created and deleted automatically?"**
 - ⌘ **No, nodes are constant, only the pods are created.**
 - ⌘ We create VMs from a cloud provider and assign those to our Kubernetes cluster.
 - ⌘ Now, the **sched** will schedule the *pods* to the available nodes only if it is up.
- Now, the question may be, if the nodes are directly assigned to the cluster, then **what is the use of c-c-m to talk to cloud provider?**
 - ⌘ One of the use-case can be creating Load Balancer (ex: AWS ELB)
 - ⌘ The flow is like the below:
 - ⌘ User will trigger request to the Load Balancer (provided by Cloud Provider, Ex: AWS ELB).
 - ⌘ Load Balancer doesn't know about the pods inside which the containers running our application are present.
It only knows about the nodes. Because the nodes are VMs of the same Cloud Provider only. It can forward the request to those.
 - ⌘ Now, lets say the Load Balancer sends the request to one node (Worker Node of course).

- ⌘ Now, **k-proxy** is there inside the worker node which will decide to which pods the request should go to.

Means, there can be multiple pods running inside the same Node, here **k-proxy** does its work.

- ⌘ **User → Cloud Provider's Load Balancer → Worker Node (VM) → Pods**

- ⌘ The user's request doesn't go to control plane. Because Control Plane is only for managing the data plane, not for user's traffic.

- ⌘ Other use cases of **c-c-m** are like

- ⌘ Storage (volumes)
- ⌘ Node lifecycle and metadata
- ⌘ Cloud networking (like pods from different nodes can communicate with each other)

- ⌘ In simple words, **CCM lets Kubernetes use cloud resources like storage, networking, and node info — not just load balancers.**

➤ -

Kubernetes Standard Interfaces

- **interface** : A standard contract (set of rules/API) that defines *how two systems talk to each other*.

- ⌘ It just defines *how to communicate*.

- ⌘ Interfaces allows Kubernetes to be *modular*.

Without this, Kubernetes would have tied to only Docker, only one network, only one storage.

- ⌘ With interfaces, you can plug anything (just like Spring Boot, you can have loose coupling when you use interfaces)

Need	You can use
Container runtime	containerd / CRI-O
Networking	Calico / Flannel
Storage	AWS EBS / GCP Disk
Cloud	AWS / Azure / GCP

- ⌘ When Kubelet wants to run a container, it doesn't say use *Docker* to run the container.

Rather it says, use CRI (Container Runtime Interface) to run the container.

- ⌘ In one sentence: Interfaces in Kubernetes are **standard ways of communication** that allow it to work with **different container runtimes, networking systems, storage providers, and cloud platforms** *without being tightly coupled to any one of them*.

- ⌘ The interfaces are: **CRI, CNI, CSI, CCM**

- **CRI (Container Runtime Interface)**

- ⌘ Interface between *kubelet* and *container runtime*.

- ⌘ It defines

- ⌘ How to *run containers*

- ⌘ How to *stop containers*

- ⌘ How to *check container status*

- ⌘ *kubelet* uses CRI.

It doesn't run the containers directly, rather it talks via CRI.

- ⌘ One question might be arises, *kubelet* works at *Pods* level, then **how it interact with containers directly?**

- ⌘ *pod* is assigned to a Node.

- ⌘ **kubelet** present inside that Node will see the Pod details (**pod is just a logical unit**).
- * Image
- * Ports
- * Number of containers
- ⌘ kubelet calls CRI
 - * It just tells the specific runtime (if docker then docker runtime) to create the containers.
- ⌘ *container runtime* does the work like pull images, create and start container.
- ⌘ **kubelet** keeps watching the containers, if crashes then tells the **runtime** to restart the containers.
- ⌘ So, **kubelet** doesn't start or create the containers, rather it just get the data from pod specs and tells *container runtime* to create and start the containers.
- ⌘ [runtime pulls images, create containers, start containers, stop/delete containers; environment(ex: linux) is provided by host OS]

⌘ NOTE:

- ⌘ CRI is an interface, and its interface must be there for the runtimes.
- ⌘ Its **not** that, ~~**kubelet will use an implementation of CRI to talk to runtime.**~~ Rather, **runtime will have CRI implementation within it.**
- ⌘ **kubelet** will send the **same type of request** to all the runtime, so it looks like all the **runtimes** are accepting same type of request.
But no, the **CRI's implementation** makes it look like that.
- ⌘ **Kubelet sends the same CRI requests to any runtime, Each runtime translates those requests internally in its own way (CRI's implementation does this translation).**

⌘ Docker doesn't have the CRI's implementation.

- ⌘ So, Kubernetes used to use one adaptor called **dockershim** which was containing the CRI's implementation.

Kubelet → CRI → dockershim → Docker → runs container

- ⌘ This **dockershim** was an extra layer to provide CRI to the docker runtime.
- ⌘ Docker has and always had its runtime called **containerd**.
[containerd is not only for docker, it works with all OCI standard containers]
- ⌘ But, **containerd** was not having CRI support before, so **dockershim** was used.

Now, **containerd** is provided with CRI support, so there is no need of using *dockershim*.

➤ CNI (Container Network Interface)

⌘ Interface between Kubernetes and Networking system.

[here Kubernetes means the **kubelet** running inside each worker node]

⌘ It defines

- ⌘ How to *attach a Pod to network*
- ⌘ How to *assign IP*
- ⌘ How to *connect it to cluster network*

⌘ **NOTE: CNI replaces Docker networks in Kubernetes. No docker network is used here.**

⌘ **kubelet** calls CNI when a Pod is created or deleted.

⌘ The flow:

- ⌘ Pod scheduled to a Node by **sched**.
- ⌘ **kubelet** starts the Pod (creates container via CRI)
- ⌘ **kubelet** calls the **CNI plugin**
 - * It sends Pod namespace info, container ID, Network config.
- ⌘ CNI plugin does networking.
 - * Depends on plugins like Calico, Flannel, ..etc, but generally it does the below
 - * Creates network interface (virtual eth pair [veth])

Pod <----veth----> Node
 - * Assigns IP to Pod.
 - * Connects Pod to cluster network.
 - ⌘ Adds routes/bridges/overlay.
 - * Update routing rules (so other pod can reach it)

⌘ In simple sentence, **CNI is the interface that allows Kubelet to configure networking for Pods by assigning IPs, creating network interfaces, and connecting them to the cluster network using plugins.**

⌘ In context of AWS, how VPC and CNI work together and how K-Proxy work in this case?

- ⌘ First thing is, CNI is there to **assign IP** to the pods where as K-Proxy is there to **select the pods to which the request will be forwarded.**

Both have different purpose.

- In AWS, each EC2 instances gets a set of IP from the VPC subnets like
10.0.0.10, 10.0.0.11 ... (Node-1)
10.0.0.20, 10.0.0.21 ... (Node-2) ..etc
- * And from these set of IPs, the pods will get one one IP.
- * The pods should not be called using their IPs directly, because Pods are dynamic and can be deleted and created depending upon the requirements.
So, it should be called using the services only.

• User → ELB → EC2 (worker node) → Pods

- Kube-proxy continuously watches the cluster and updates iptables rules.

Actual traffic forwarding is done by the OS using those rules

Its like, K-Proxy writes the rulebook, Kernel execute that.

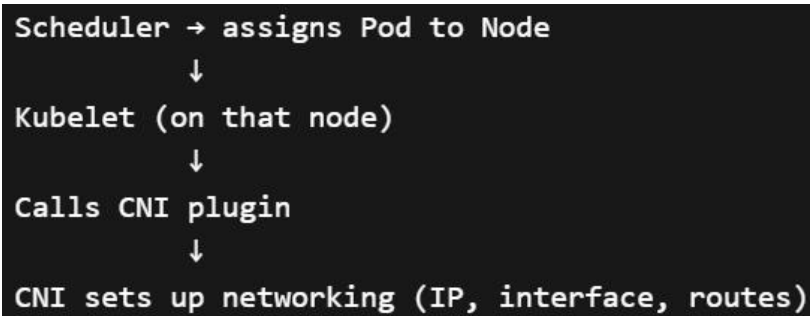
- One more thing, Load Balancer is not part of the Kubernetes cluster. But the K-Proxy is.

User ----- (EC2)Node's port (outside of kubernetes cluster)

After getting into the node, the K-Proxy will route to the matching pod
and you can access the application.

Matching pod means, if the request is for order service, and multiple services are running on different pods like payment service, inventory service, then the request will be forwarded only the pods running order service.

- User → Load balancer → Node → K Proxy → Pod



Kubelet calls CNI plugin.

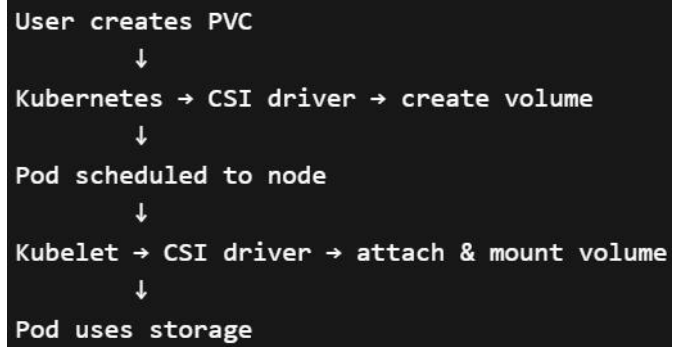
➤ CSI (Container Storage Interface)

- In this, both Control plane + Kubelet are involved.

Control plane uses CSI to create or delete volumes.

Kubelet uses CSI to attach disk to node, and mount to pod.

- Kubernetes doesn't manage storage itself, It delegates to CSI drivers.



⌘ PVC: PersistentVolumeClaim

⌘ In simple terms, **CSI is the interface that allows Kubernetes to interact with external storage systems to create, attach, and mount volumes for Pods.**

➤ -

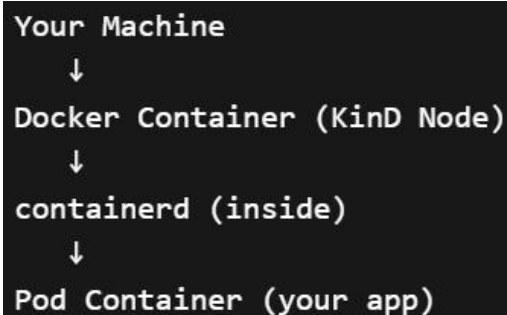
Installing and Setup (KinD, Civo, GKE)

➤ KinD

- ⌘ It is used to run the Kubernetes cluster inside Docker container without involving any cloud.
It is just used for learning, testing, CI pipelines.
- ⌘ You can create as many containers as you want. All containers will be treated as a Node.

```
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

- ⌘ Lets say you creates only one container, and install all the services of both control plane and data plane inside that.
Then just like the single node setup, the **sched** will assign all the pods to the same node (the container that is running all services).
- ⌘ Basically the container runs *Kubelet*, *containerd* (container runtime), kube-proxy, and *control plane components* (if it's a control plane node).



- ⌘ In simple words: **KinD** uses Docker containers as *fake VMs* to simulate Nodes.
- ⌘ **Every Kubernetes cluster MUST have control plane.**
At least one KinD container must be a control-plane node. Worker (data plane) containers are optional
- ⌘ It looks like container inside container (when pods runs the container within it, but the pod is there inside KinD container)
 - ⌘ Containers are nothing but a namespace and cgroups.
At kernel level, it is just a process.
 - ⌘ So, when kubelet will call containerd (inside KinD container (node)) it'll create a new container.
It looks nested, but its actually not. The new container will have a new set of namespace and cgroups.

- ⌘ At kernel level, both the containers (KinD container, container inside Pod) are same only (both are having an unique combination of namespace and cgroups).

But from outside, it looks nested.

➤ **Civo**

- ⌘ It is a cloud provide (like AWS, Azure, GCP) which provides managed Kubernetes (and cloud infrastructure).
- ⌘ It provides managed Kubernetes cluster, networking, virtual machines, storage.

➤ **Nix**

- ⌘ It is a package manager that just download the dependencies directly in your machine of any version you specifies but in a specific directory.
- ⌘ Its similar to docker in context of keeping dependencies at some directory but it doesn't have any containers.
- ⌘ **DevBox** is a wrapper written on the top of Nix to make it simpler.
- ⌘

Tools for Simplicity (not related to Kubernetes)

➤ Devbox

- ♣ It is a wrapper around **Nix** to make the things simple.
- ♣ Create a json file, and write all the dependencies there.
Nix installs packages in its *own store*.
- ♣ when you run **devbox shell**, it'll open a shell where all the dependencies which are installed will be available and you can use those (kind of python virtual env).

Python venv	Devbox
Python-only	Any tool (Node, Go, Terraform, etc.)
Local install	Uses Nix store (shared, reproducible)

- ♣ In simple words, **Devbox** is a wrapper over **Nix** that lets you define project dependencies in a config file and provides an *isolated shell environment where those dependencies are available*, similar to a virtual environment but for any kind of tool.
- ♣ **json** file will contain the list of dependencies, **lock** file will contain the versions of those dependencies.
- ♣ If you are running **Devbox** in windows, then always choose the *WSL* shell, not git bash or cmd or powershell.
In case of Mac or Linux, you can directly run.
- ♣ **Only inside devbox shell you can use those installed dependencies, not outside.**
Its completely isolated from the other dependencies running inside your system.

➤ Taskfile

- ♣ Taskfile is a YAML-based way to define and run project tasks (like scripts/commands)
- ♣ Example:

```
version: '3'

tasks:
  build:
    cmds:
      - docker build -t app .

  run:
    cmds:
      - docker run -p 3000:3000 app
```

- ✧ Instead of writing the commands again and again, you can just write like this

task build

task run

(just like package.json in nodejs, npm start, npm build ..)

- ✧ So, we'll use **Devbox** to install dependencies and open a shell which is linked to all those dependencies.

And, **Taskfile** to execute the reusable commands instead of writing those commands manually.

```
version: "3"

env:
  CLUSTER_NAME: devops-directive-kubernetes-course
  CIVO_REGION: NYC1
  GCP_REGION: us-central1
  GCP_ZONE: us-central1-a
  # Set default gum style options
  BORDER: double
  BORDER_FOREGROUND: "212"
  PADDING: "1 1"
  MARGIN: "1 1"

tasks:
  kind:01-generate-config:
    cmds:
      - REPLACE_WITH_ABSOLUTE_PATH=${PWD} envsubst < kind-config.yaml.TEMPLATE > kind-config.yaml
    desc: "Generate kind config with local absolute paths for PV mounts"

  kind:02-create-cluster:
    cmds:
      - kind create cluster --config kind-config.yaml
    desc: Create a Kubernetes cluster using kind

  kind:03-run-cloud-provider-kind:
    desc: "Run sigs.k8s.io/cloud-provider-kind@latest to enable load balancer services with KinD"
    cmds:
      - sudo cloud-provider-kind

  kind:04-delete-cluster:
    cmds:
      - kind delete cluster
    desc: Delete and existing a kind Kubernetes cluster
```

(example of a TaskFile.yaml)

- ✧ When you execute, **task --list**, it'll display all the tasks you have configured inside this file. For example

kind:01-generate-config

kind:02 ... etc etc

```
* kind:01-generate-config:      Generate kind config with local absolute paths for PV mounts
* kind:02-create-cluster:      Create a Kubernetes cluster using kind
* kind:03-run-cloud-provider-kind: Run sigs.k8s.io/cloud-provider-kind@latest to enable load balancer services with KinD
* kind:04-delete-cluster:      Delete and existing a kind Kubernetes cluster
```

➤ Kind config

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
  extraMounts:
  - hostPath: /mnt/c/my files/courses_and_projects/gitlab/kubernetes
    containerPath: /some/path/in/container
- role: worker
  extraMounts:
  - hostPath: /mnt/c/my files/courses_and_projects/gitlab/kubernetes
    containerPath: /some/path/in/container
```

- ⌘ You can see the task kind:02-create-cluster in the previous image, that was using this yaml file.
- ⌘ 3 containers (here nodes) are created. One will be having control plane and other 2 will be worker nodes.

```
task: [kind:02-create-cluster] kind create cluster --config kind-config.yaml
Creating cluster "kind" ...
  ✓ Ensuring node image (kindest/node:v1.30.0)
  ✓ Preparing nodes
  ✓ Writing configuration
  ✓ Starting control-plane
  ✓ Installing CNI
  ✓ Installing StorageClass
  ✓ Joining worker nodes
Set kubectl context to "kind-kind"
```

- ⌘ Execute the command **kubectl get nodes** to see the list of nodes.

NAME	STATUS	ROLES	AGE	VERSION
kind-control-plane	Ready	control-plane	66s	v1.30.0
kind-worker	Ready	<none>	45s	v1.30.0
kind-worker2	Ready	<none>	45s	v1.30.0

- F
- F
- F
- F
- F
- F
- F
- F
- F

- F
- F
- F
- F
- F
- F
- F
-